Does the Representation of Formal Verification Feedback Matter for LLM-Based Compiler Repair?

Study design and an IR-aware counterexample reducer for real LLVM middle-end bugs

Abstract

Large language models can repair real bugs in LLVM's middle end, and the Alive2 translation validator can prove whether a proposed optimisation is semantics-preserving and, when it is not, emit a concrete counterexample. Existing repair loops feed that counterexample back to the model essentially as the verifier printed it. Alive2 counterexamples on real bugs are not small: they are drawn from real reproducers carrying vector types, metadata, attribute groups, and dozens of instructions unrelated to the violation. The signal is present but diluted.

We ask whether the *representation* of that feedback materially affects repair — specifically, whether LLVM-IR-aware, semantics-preserving minimisation of the counterexample, and structured rendering of it, constitute a stronger signal than generic textual reduction or raw verifier output. To isolate that question, the entire repair loop is held fixed and exactly one stage is varied: the text handed to the model after a failed verification. Conditions form a 4×2 grid (reduction $\times$ representation) plus a no-counterexample control.

This paper contributes the method and the design. (i) An **IR-aware reducer** that edits source and target modules in tandem, closes each candidate edit over its dependencies, and seeds its search from the violation the verifier reported, with every edit gated by a semantic-preservation oracle at selectable strictness. (ii) **Two deliberately contrasting baselines**: a text-level line reducer that is IR-blind, isolating prompt length from IR awareness, and a wrapper around LLVM's own `llvm-reduce`, which is IR-valid by construction but counterexample-blind. (iii) A **structured feedback renderer** that decomposes a counterexample into labelled sections — inputs, source result, target result, violation class, interpretation. (iv) A **reproducible study population**: a deterministic, complexity-tiered, component-diversified 24-bug sample drawn from the 99 benchmark issues this harness can actually attempt, together with an IR reader validated to round-trip all 1,462 real reproducers in the benchmark byte for byte. (v) A **preregistered analysis plan** fixing the repeat count, the paired test, its one-sided direction, the multiple-comparison correction, and exactly which four comparisons are primary — including an explicit statement of how little the design can resolve: even under optimistic effect sizes its power caps near 57 %, so it is a screen rather than a confirmation.

No repair-rate data are reported. The experiment is pending.

1 Introduction

1.1 The problem

A miscompilation in an optimising compiler is a bug with unusual reach: it silently changes the meaning of correct source programs, and every program built by that release inherits the defect. LLVM's middle end — the target-independent optimisation pipeline containing InstCombine, the loop passes, SCEV, the vectorisers — is where most of these live, because that is where the most aggressive semantic reasoning happens.

Two developments make automated repair of such bugs plausible.

First, **translation validation**. Alive2 [1] takes the IR before and after an optimisation and attempts to prove the transformation refinement-correct under LLVM's semantics, including undefined behaviour and poison. When the proof fails it emits a **counterexample**: concrete input values, the source function's result, the target function's differing result, and a classification of the violation — a value mismatch, an illegal poison introduction, a memory-state divergence. That is a far richer failure signal than a crashing test case.

Second, **LLM-based program repair** has reached the point where models propose plausible patches to real compiler source given a bug report and a window of the offending code [3, 4].

Combining them yields the loop this study inherits: show the model the bug, take its patch, rebuild the compiler, run the regression tests, verify with Alive2, and on failure feed something back and iterate.

1.2 The question

That loop's feedback step is, in existing systems, close to a pass-through — the verifier's output is inserted into the next prompt more or less verbatim. Generic program-repair research suggests this matters: reduced and better-

structured failure information improves LLM repair in other settings [5]. Compiler-specific repair systems have shown that agents can fix real LLVM bugs [6, 7]. Neither line of work has asked whether *LLVM-semantics-aware* minimisation of a formal counterexample beats generic input reduction as a feedback signal.

So:

Does the representation of formal verification feedback materially affect LLM-based compiler repair, and does LLVM-specific semantic awareness outperform generic input reduction?

The intended causal chain, stated plainly so that the design can be checked against it: better formal feedback → better model understanding → more correct patches → fewer repair iterations → lower cost → more reliable automated compiler repair. The study tests the first two links and measures the third and fourth.

Decomposed into the questions the design is built to answer (Table 1):

Table 1: Research questions.

RQ1	Can an IR-aware reducer substantially shrink Alive2 counterexamples while provably preserving the semantic violation?
RQ2	Does a reduced counterexample raise the probability that the model produces a correct patch?
RQ3	Does IR-aware reduction outperform generic and IR-valid-but-counterexample-blind reduction?
RQ4	Does minimisation reduce **repair iterations** — and with them rebuilds and verifier calls?
RQ5	Does structured semantic annotation improve repair beyond the raw reduced counterexample?
RQ6	Is the best performance obtained by combining minimisation with structured annotation, rather than either alone?

RQ4 needs its scope stated, because it is easy to overclaim. Each repair iteration means a **full LLVM rebuild** — minutes to hours, not the microseconds a prompt-token difference costs. Saving a few hundred prompt tokens is noise against a rebuild. The efficiency claim is therefore stated in **iterations**, and in the build and verifier counts that scale with them. Tokens and wall-clock time are recorded but are *descriptive context only*, never a headline efficiency figure. The token estimator is characters ÷ 4 — adequate for comparing conditions, wrong for absolute cost.

1.3 What is and is not new here

Three adjacent things are already published and must not be claimed:

- **Not new:** that LLMs can repair real LLVM bugs. Demonstrated by prior compiler-repair systems [6, 7].
- **Not new:** reducing a failure-inducing input. That is delta debugging and its descendants [2], and `llvm-reduce` is LLVM's own production implementation.
- **Not new:** that smaller, cleaner failure information helps LLM repair in general [5].
- **Proposed here:** that *LLVM-IR-aware, semantics-preserving* minimisation plus *structured rendering* of an Alive2 counterexample is a materially better feedback signal for compiler repair than generic reduction or raw verifier output — and a controlled design capable of testing that.

The repair loop is deliberately **not** a contribution. It is an existing benchmark's own simple baseline loop with the feedback call swapped, precisely so that any difference between conditions is attributable to the feedback rather than to agent design.

1.4 Contributions

1. **An IR-aware counterexample reducer** (§4.4) that edits source and target modules in tandem, closes each candidate edit over its dependencies, seeds its search from the violation reported by the verifier, and accepts an edit only when a semantic-preservation oracle confirms the same bug survives.
2. **Two contrasting baselines** (§4.4): a text-level line reducer that is deliberately IR-blind, separating "the prompt got shorter" from "the prompt got semantically sharper"; and a wrapper around LLVM's own `llvm-reduce`, IR-valid by construction but counterexample-blind, which answers the objection *wouldn't any real reducer have done this?*
3. **A structured feedback renderer** (§4.5) that decomposes a counterexample into labelled sections rather than plain verifier text.
4. **A reproducible study population** (§3): a deterministic, complexity-tiered, component-diversified 24-bug sample drawn from the 99 benchmark issues this harness can actually attempt, plus an IR reader validated against all 1,462 real reproducers.
5. **A preregistered analysis plan** (§6) fixing $k = 3$, McNemar's exact test one-sided, Benjamini–Hochberg over exactly four primary comparisons, the reporting rules that go with them, and an explicit account of the design's resolution limits.

2 Background

2.1 Miscompilation and translation validation

An optimisation is correct if the transformed program *refines* the original: it may be more defined, never less, and must agree wherever the original is defined. LLVM's se-

mantics make this subtle, chiefly through `undef`, `poison`, and the fast-math and no-wrap flags that license a pass to assume a condition the source never guaranteed.

Alive2 discharges refinement obligations with an SMT solver over a bounded unrolling of the two functions [1]. Its value for repair is not the pass/fail bit but the **counterexample**: the concrete witness that separates source from target. A repair loop that only knows "tests still fail" is guessing; one that knows *which inputs* produce *which divergent value*, and *which class of violation* that is, has something to reason from.

2.2 Benchmarks of real LLVM bugs

This study builds on an existing LLVM APR benchmark, used unmodified as a vendored subdirectory. It supplies real issues with base commits, reproducers, hint regions locating the likely fault, a build-and-test harness, and verification dispatch.

Counts in Table 2 are taken live from the vendored dataset rather than quoted from the benchmark's own README, which is stale.

Table 2: Benchmark composition, counted from the 491 dataset files at analysis time.

	Count
Issue files in dataset	491
— miscompilations	142
— crashes	340
— hangs	9
Reproducer bodies across all issues	1,462
Single-function fixes (all bug types)	373
Single-function-fix miscompilations	103

The knowledge-cutoff fields in the dataset span 2024-01 to 2026-02, which is the benchmark's mechanism for contamination control (§6.4).

2.3 LLM-based program repair, and compiler repair specifically

General LLM-based APR has converged on iterative loops with execution feedback: propose a patch, run the tests, feed the failure back, repeat [3, 4]. The design variables that have mattered most are the fault-localisation signal handed to the model, the failure information fed back, and the iteration budget.

Compiler repair is a harder instance on three axes. The codebase is large and domain-specialised; the oracle is expensive, because evaluating a patch means rebuilding the compiler; and correctness is not exhausted by tests, since a patch can pass the regression suite and still be a miscompilation — which is exactly why translation validation belongs in the loop.

2.4 Input reduction

Delta debugging and its successors minimise failure-inducing inputs by systematically removing parts and re-testing [2]. `llvm-reduce` is LLVM's production reducer for IR: it understands IR structure and preserves validity, and it is driven by an arbitrary "interesting-ness" predicate.

Reduction has also been applied to improve repair feedback in general APR settings [5]. The gap this study targets is the intersection none of these occupies: reduction aware of *both* IR semantics *and* the specific violation the verifier found. `llvm-reduce` preserves IR validity but knows nothing about the counterexample; delta debugging over text knows neither.

3 Study population

3.1 From 491 issues to 24 bugs

Not every benchmark issue is attemptable by this harness. Alive2-based verification applies to miscompilations; and the repair runner skips anything that is not a single-function fix, because the prompt shows the model one source window. Applying both filters, the usable population is **99 issues**, not 491 — a distinction that will govern every rate reported from this design. **Repair rates will be rates over the miscompilation subset, never over the whole benchmark.**

Running all 99 bugs × 9 conditions × k repeats, at up to four LLVM rebuilds per attempt, is thousands of builds at up to hours each. The study therefore uses a subsample, chosen and justified in advance rather than however far a sweep happens to get before someone stops it.

Stratification is two-dimensional, in priority order.

Complexity tier (primary). Bugs are scored by hint-region size and reproducer size and split into three equal tiers — easy, medium, hard. This is the dimension that plausibly interacts with the research question: if IR-aware reduction's advantage appears only on complex bugs, or only on simple ones, a sample skewed toward one tier would hide it.

Component diversity (secondary, within tier). 45 of the 99 qualifying bugs are InstCombine, so a naive "simplest N overall" sample would be mostly InstCombine at every tier. Within each tier, selection maximises distinct components before repeating any.

Selection is fully deterministic — score, then component, then bug ID as final tiebreak, with no RNG — so the sample is reproducible from the dataset alone and citable as an exact list rather than "some random sample". Re-running the selector for this paper reproduced the committed 24-bug manifest identically, with the same pool size of 99. One bug is excluded by construction: the bootstrap bug used during harness development, which

would otherwise be the one bug the implementation had been tuned against.

Figure 1 shows the filtering and the resulting composition.

The sample size of 24 (8 per tier) is a judgement call, not a statistical derivation: small enough to be tractable at hours per build, large enough to say something about a nine-condition comparison. §6.3 quantifies exactly how little it can resolve, which is part of the design rather than a caveat on it.

3.2 Validating the IR reader

The reducer rests on one safety rule: **anything the IR model does not understand must survive parse-then-print unchanged.** Real reproducers contain vector types, metadata, attribute groups, `byval`, and much else the model never explicitly represents. If that rule broke, the reducer would silently corrupt code it only meant to shrink, and every downstream measurement would be garbage.

The rule is tested against every real reproducer in the benchmark, and re-run for this paper: **491 issues, 1,462 reproducers, 1,462 round-tripped byte-identically, 0 mismatched, 0 crashed.** The check earned its keep during development, catching dropped blank lines, dropped comments, and re-indentation of column-zero instructions.

3.3 Build feasibility

Each bug must be built at its own base commit, because a patch is evaluated against the compiler as it stood when the bug was filed. All 24 sample bugs have a build-verified `opt` on the development machine, built in five batches; per-bug wall time ranged from ∼18 minutes to ∼2 h 49 m depending on compiler-cache hit rate, roughly 20 hours of sequential compute in total. Builds do not transfer between machines — the build and cache volumes are per-machine — so reproduction elsewhere re-pays this cost.

One practical hazard is worth recording for anyone reproducing the setup. With one compile job per core, LLVM's heaviest translation units exceeded the development container's memory cap and were OOM-killed 10–20 minutes into the build, reported identically to a genuine build failure; all ten bugs in the first batch failed this way before the cause was found. Lowering build parallelism to four jobs fixed it, and the compiler cache was raised from its 5 GB default to 40 GB, since the default could not hold much across builds for 24 different base commits.

4 Method

4.1 What is varied, and what is not

The repair loop is held fixed; exactly one thing varies: **the text handed to the model after a failed verification.** The runner is the benchmark's own baseline loop with the feedback call swapped. The iteration budget (four model turns per bug) and the oracle budget (400 verifier calls per reduction) are identical across all conditions and exposed as flags, so no condition can win by being allowed more attempts or more verifier time. Figure 2 shows the loop and the condition grid.

4.2 Conditions

Nine conditions: the 4×2 grid plus the control (Table 3).

Table 3: The nine experimental conditions.

Condition	Reduction	Repr.	CEX
`baseline`	—	—	no
`raw-plain`	none	plain	yes
`generic-plain`	text-level	plain	yes
`llvmreduce-plain`	llvm-reduce	plain	yes
`iraware-plain`	IR-aware	plain	yes
`raw-structured`	none	structured	yes
`generic-structured`	text-level	structured	yes
`llvmreduce-structured`	llvm-reduce	structured	yes
`iraware-structured`	IR-aware	structured	yes

The `baseline` control is what an existing repair loop does: the bug, the source window, and the build/test outcome, with no counterexample at all. It establishes whether showing a counterexample helps *in any form* before the design asks which form is best.

A naming caution. The project's framing document letters these conditions in two mutually contradictory ways, and the implementation follows only one of them. Condition names are therefore always written out in full and never abbreviated to bare letters; anyone cross-reading the repository's internal documents should do the same.

4.3 The semantic-preservation oracle

Shrinking a counterexample is trivial if correctness is irrelevant — delete everything and the file is very small indeed. What makes shrinking *useful* is that the result still demonstrates **the same compiler bug**. No reducer here deletes and hopes. Each one:

1. proposes an edit — delete an instruction, drop a flag, prune a block;
2. re-runs Alive2 on the result;
3. asks whether the same bug is still there;

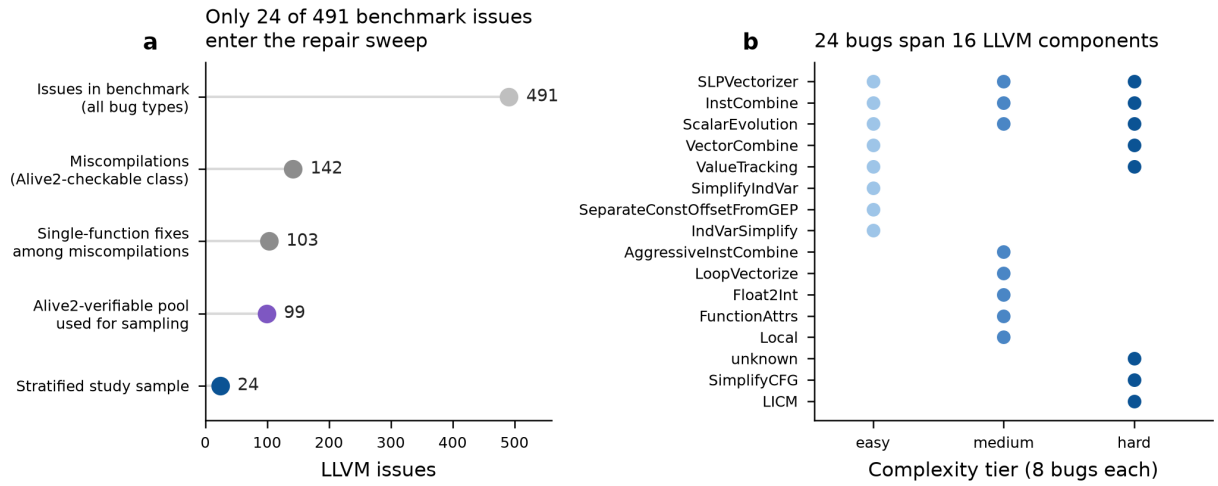


Figure 1: Study population. **a**, Filtering from the full benchmark to the sweep sample; each stage is a single count, so stages are drawn as points rather than bars. The Alive2-verifiable pool (99) is the selector's candidate set after both the single-function-fix and verifier-checkability filters, and is therefore smaller than the 103 single-function-fix miscompilations. **b**, Composition of the 24-bug sample: 16 distinct LLVM components across three equal complexity tiers, with points offset horizontally where a tier contains more than one bug from the same component.

and accepts the edit only on a yes. The oracle is therefore the most important component in the design: it is what converts "smaller" into "smaller and still valid evidence".

"The same bug" is a choice, and the oracle exposes three strictness levels (Table 4).

Table 4: Oracle strictness levels.

Level	Accepts an edit when / risk
`any_failure`	any verification failure remains. Weakest — permits drift onto a *different* bug in the same function.
`error_class` (default)	the failure is of the same class. Balanced.
`error_class_and_kind`	class and kind both match. Strictest, fewest accepted edits.

Because the weakest level permits drift, **the strictness in force must be reported alongside any reduction figure**, and figures are not comparable across levels. The default for this study is `error_class`.

Two further guards. The oracle is budgeted, so a reducer cannot spend unbounded verifier time; and if any edit was applied, the final rendered pair is re-checked, with the original returned untouched on failure. No reducer hands back a pair the oracle has not signed off on.

4.4 The three reducers

`generic` — text-level, deliberately IR-blind. Line-level delta debugging over the IR text. It exists as a control, not as a serious competitor: if IR-aware reduction wins, `generic` is what separates "the prompt got shorter" from "the prompt got semantically sharper". It is expected to perform badly on IR, because most line deletions produce invalid IR, which the oracle rejects.

`llvmreduce` — LLVM's own reducer, counterexample-blind. A wrapper around `llvm-reduce`, applied to source and target. IR-valid by construction and a genuinely strong reducer. It was added deliberately to strengthen the design, because the `generic` baseline alone is too weak to be interesting: without `llvmreduce`, a win for the proposed method would be open to the objection that *any* real IR reducer would have achieved it. What `llvm-reduce` does not know is *which* violation matters.

`iraware` — the proposed method. Operates on a parsed IR model rather than on text, and differs from both baselines in three ways that carry the hypothesis:

1. **Source and target are edited in tandem.** A counterexample is a *pair*. Editing one side alone generally destroys the comparison the verifier is making, which is why text-level reduction fails on this input class.
2. **Candidate edits are closed over dependencies.** Removing an instruction requires removing or rewriting everything that depends on it, so candidates are whole dependency-closed groups rather than single lines. Accepted edits are consequently large, and few oracle calls are needed per unit of reduction.
3. **The search is seeded from the violation.** A backward slice is taken from the values the verifier reported as mismatching, together with the function's returned values. The reducer looks where the bug is

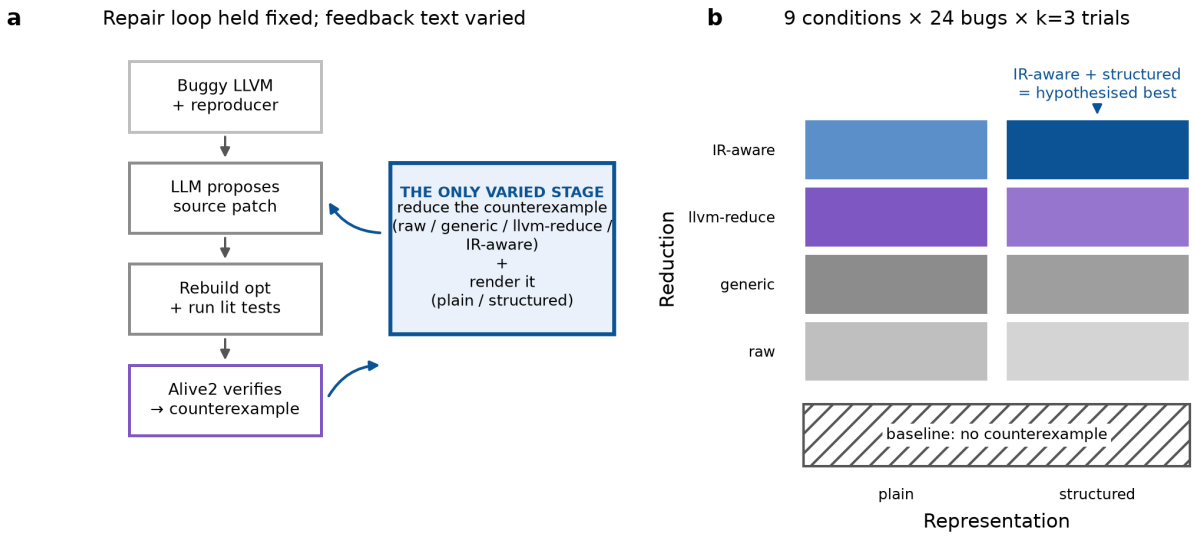


Figure 2: Design. **a**, The inherited repair loop. Every stage is held fixed; the only varied stage is the construction of the feedback text after a failed Alive2 verification. **b**, The condition grid: four reduction levels × two representations, plus a no-counterexample control, run over 24 bugs at $k = 3$ trials each.

rather than searching the module uniformly. This is the one piece of information `llvm-reduce` structurally cannot use.

Its pass list covers dead-code elimination, slicing, branch folding, block pruning, CFG simplification, parameter dropping, flag stripping, and an **operand-promotion** pass that generalises concrete operands into parameters. Operand promotion is a distinct kind of change — it generalises rather than deletes — so it is separately switchable, giving it its own ablation axis instead of folding its contribution silently into the headline reduction.

4.5 Representation

`plain` renders the counterexample essentially as the verifier printed it — the format an existing loop would pass through.

`structured` decomposes the same content into labelled sections: the concrete inputs, the source function's result, the target function's differing result, the violation class, and a short interpretation of what that class means. For a poison violation, for instance, the interpretation states that a transformation may only make a program *less* poisonous, so attaching `nsw` in the target where the source has none is illegal on its face.

The interpretation text is templated per violation class, not generated, so it is identical across bugs within a class and cannot introduce a per-bug confound. Representation is orthogonal to reduction by construction, which is what makes RQ6 — the interaction — askable at all.

4.6 Model serving and the repair runner

The model is served locally through an OpenAI-compatible endpoint so that temperature, context length, and served model name are under experimental control rather than a vendor's. Sampling temperature must be **greater than zero**, or k repeats buy k copies of one answer (§6.1).

Two serving parameters required deliberate setting rather than defaults, and both had already caused silent failures during pilot work: context length, because a too-small window truncates the longest condition's prompt while looking exactly like genuine repair failure; and the memory fraction and eager-execution settings, which interact with weight loading and cache budgeting on a shared accelerator. The intended primary model did not fit alongside a standing tenant on the shared accelerator, so a smaller model from the same family was substituted; the substitution is recorded as a deviation from the preregistered choice rather than quietly adopted.

The runner is resumable and persists its environment, because a full sweep is a multi-day job with thousands of compiler rebuilds and cannot be assumed to run uninterrupted. Each run records the full conversation, per-iteration feedback dictionaries, patch outcome (applied / unchanged / mismatch), build and verifier counts, and timing — so that a failure can be diagnosed after the fact rather than re-run.

5 Experimental design

5.1 Procedure

For each of the 24 sample bugs, for each of the 9 conditions, for each of $k = 3$ trials:

1. Check out LLVM at the bug's base commit and build `opt`.
2. Present the model with the bug type, issue title, issue body, reproducer, and the source window around the hint region, together with the format requirement.
3. Apply the returned patch to the source window.
4. Rebuild `opt`; run the relevant lit test directory.
5. Run Alive2 translation validation.
6. On failure, construct the feedback text **according to the condition** — reduce the counterexample at the condition's reduction level, render it at the condition's representation level, or omit it entirely for `baseline` — and return to step 2.
7. Stop on success or after four iterations.

The source window is selected identically under every condition, and is grown until its running brace depth never dips below zero and finishes at zero, so the model receives complete syntactic units rather than an excerpt starting mid-statement. A reply identical to the window is rejected as a no-op before a rebuild is spent on it, and the model is told it returned the code unchanged. Both behaviours are uniform across conditions, and both are deviations from the inherited baseline loop, recorded as such: the claim that *nothing* besides the feedback text differs from the upstream loop does not hold, and saying so is part of the method.

5.2 Measures

Table 5 lists what each run records and the role each measure plays.

Table 5: Measures and their role. Tokens and wall-clock time are never an efficiency claim.

Measure	Definition / role
Fixed	patch applies, builds, passes lit tests, and Alive2 verifies. Primary outcome (RQ2, RQ3, RQ5, RQ6).
Iterations to fix	model turns consumed before success. Primary efficiency measure (RQ4).
Builds, **Alive2 calls**	totals per run. Corroborating efficiency detail (RQ4).
Reduction ratio	instructions removed ÷ instructions in. Reducer measure (RQ1, RQ3).
Oracle calls	verifier calls consumed by reduction. Reducer cost (RQ3).
Tokens, **wall-clock**	recorded per run. Descriptive context only.

6 Preregistered analysis plan

The plan was committed **before any repair-rate data existed** — the results directory contained only build transcripts at the time — as its own dated file rather than folded into the methodology document afterwards. That is what makes it a preregistration rather than a rationalisation. Nothing in it may be revised once the sweep starts.

6.1 Outcome and unit of analysis

The unit is the **bug**, not the run. At $k > 1$ the outcome is **pass@k**: a bug counts as fixed under a condition if at least one of its k trials fixed it.

Repeats do **not** add statistical units — pass@k still yields one paired binary per bug, so n stays 24 however many times each cell is re-run. What k buys is a less noisy outcome per cell. At $k = 1$ and non-zero temperature each cell is a single Bernoulli draw, so a discordant pair is as likely to be sampling noise as a real difference; noise inflates the discordant counts symmetrically, dragging their ratio toward 1. That is a bias toward **false negatives**: at $k = 1$ the experiment is set up to miss an effect that is really there.

6.2 Test, direction, correction, and the primary comparisons

Paired binary outcomes over the same 24 bugs, so **McNemar's exact test**, **one-sided** in the hypothesised direction, with **Benjamini–Hochberg** correction over exactly four preregistered comparisons:

1. `iraware-structured` vs `baseline` — does the full method beat no counterexample at all?
2. `iraware-plain` vs `generic-plain` — does IR awareness beat text-level reduction?
3. `iraware-plain` vs `llvmreduce-plain` — does counterexample awareness beat counterexample-blind IR reduction?
4. `iraware-structured` vs `iraware-plain` — does structure add anything on top of reduction?

Every other pair in the grid is **exploratory**, reported as description and **not as evidence**.

Reporting rules, fixed in advance:

- Every p-value is accompanied by the **discordant counts** and the **paired rate difference**. A p-value alone is not reportable here, because at this sample size it is dominated by a handful of bugs.
- A non-significant result means **the sample cannot resolve the question** — not that no effect exists.
- Deviations from this plan are recorded as deviations, with dates, not silently absorbed.

6.3 What the design can and cannot resolve

McNemar's exact test looks only at **discordant** bugs — those exactly one condition fixed — and asks whether their split is worse than a coin. This imposes a hard floor: **fewer than 5 discordant bugs cannot reach $p < 0.05$ one-sided, no matter how lopsided the split.**

Simulation over 20,000 replicates — each bug given a difficulty multiplier shared by both conditions, which is what the paired design exploits, and the outcome taken as pass@k — gives the power available at $n = 24$ under six plausible per-attempt rate scenarios (Table 6, Fig. 3).

Table 6: Power of the one-sided exact McNemar test at $n = 24$ paired bugs, by trials per cell. 20,000 simulations per cell. No scenario reaches conventional 80 % power.

Rates (better vs worse)	k=1	k=2	k=3	k=4	k=5
25 % vs 10 %	0.23	0.44	**0.57**	0.64	0.67
15 % vs 5 %	0.12	0.31	**0.45**	0.55	0.64
30 % vs 15 %	0.21	0.36	**0.45**	0.48	0.48
20 % vs 10 %	0.12	0.23	**0.32**	0.39	0.43
35 % vs 20 %	0.20	0.32	**0.36**	0.35	0.33
12 % vs 8 %	0.03	0.07	**0.10**	0.12	0.15

Two design decisions follow, and one limitation.

Why $k = 3$. Power roughly triples from $k = 1$ to $k = 3$ and then flattens; in the highest-rate scenario it *falls* past $k = 3$, because pass@5 pushes both conditions toward a ceiling where neither is discordant any more. $k = 3$ is where the curve stops paying for the rebuilds it costs.

Why temperature must exceed zero. At temperature zero, $k = 3$ buys three identical answers and none of the noise reduction the power table assumes.

The limitation, stated in advance. No scenario reaches conventional 80 % power; the best case is ∼57 %. **This design is a screen, not a confirmation.** A null result from it will be close to uninformative and must be reported as an unresolved question rather than as evidence of absence.

6.4 What may not be claimed

No absolute repair rate may be claimed against the benchmark leaderboard. The benchmark's contamination control relies on a model whose training cutoff precedes the bug, and no usable open-weights model can honestly claim that for these bugs. All claims are **relative comparisons between conditions under one model**, where contamination inflates every condition equally and therefore threatens only the absolute rate — which is already forbidden.

Three further prohibitions, recorded here so they bind the write-up:

- **Rates are over the attemptable miscompilation subset**, never over the 491-issue benchmark.
- **Token and wall-clock figures are descriptive only.** The efficiency claim is iterations, builds, and verifier calls.
- **Published technique, inherited infrastructure, proposed method, hypothesis, and measured result are never collapsed into one another.** The repair loop, the benchmark, and the verifier are inherited; only the reduction and representation stage is proposed; and nothing about repair effectiveness is measured yet.

7 Design limitations known in advance

These are properties of the design, not of results, and are stated here because they constrain what the experiment could show even if it ran cleanly.

Power. At most ∼57 % under optimistic effect sizes (§6.3). A screen, not a confirmation.

Single model, single temperature. One instruct model, not the preregistered primary, forced by accelerator memory pressure. Nothing in this design speaks to how feedback representation interacts with model scale or family — and that interaction is known to be real: the inherited prompt was written and validated against a frontier reasoning model and does not transfer down in scale unchanged, which is why §5.1's window and no-op handling had to be specified explicitly.

Contamination. Unquantified. Mitigated only by the prohibition on absolute-rate claims.

Oracle strictness. All reduction figures will be reported at `error_class`. A weaker setting permits drift onto a different bug in the same function, so figures are not comparable across strictness levels.

Population restriction. 24 bugs drawn from 99 attemptable issues out of 491 — miscompilations with single-function fixes. Results will not generalise to crashes, hangs, or multi-function repairs.

Build non-transferability. Per-machine build and cache volumes mean reproduction requires re-paying roughly 20 hours of compilation.

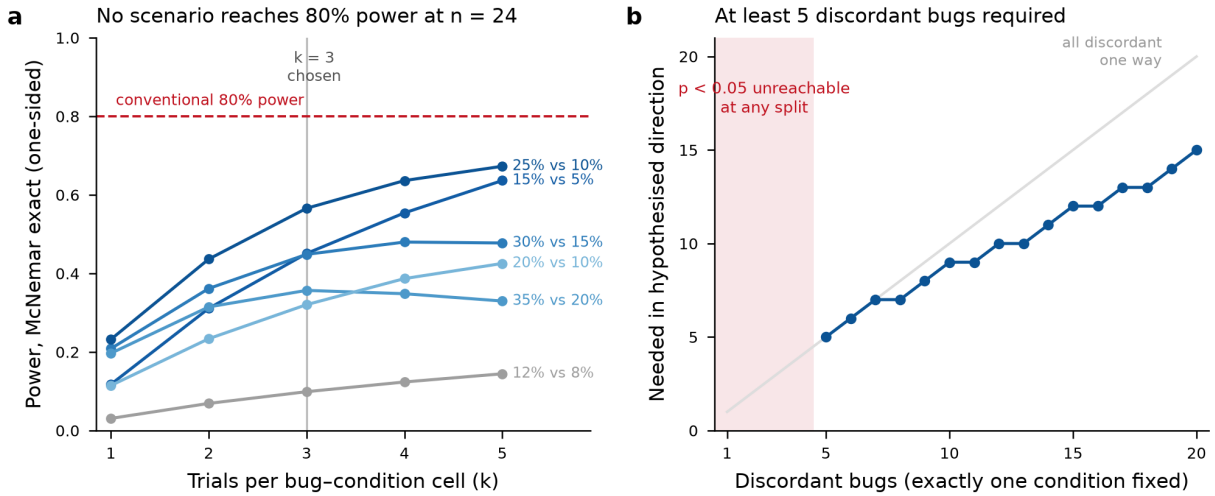


Figure 3: Statistical resolution of the design, computed before data collection. **a**, Power of the one-sided exact McNemar test over 24 paired bugs, by trials per cell, for six plausible per-attempt rate scenarios (20,000 simulations each). No scenario reaches conventional 80 % power; the 35 % vs 20 % curve declines past $k = 3$ as pass@k pushes both conditions toward a ceiling where neither is discordant. **b**, The discordance floor: with fewer than 5 discordant bugs, $p < 0.05$ one-sided is unreachable at any split. The grey diagonal marks "all discordant bugs in the hypothesised direction".

Token estimates. Characters ÷ 4 — fine for comparing conditions, wrong in absolute terms, never quoted as cost.

Multi-function modules. The reducer only touches the function containing the bug; it never removes other functions from the module.

Diagnosability of failure. Every failure mode in this harness — a truncated prompt, an unmodified reply, a malformed source window, an OOM-killed build — presents as a failed repair. The runner therefore records patch outcome, per-iteration feedback, and full conversations explicitly, so that a repair rate of zero can be **diagnosed rather than interpreted**. This is a design requirement, not an afterthought.

A Reproduction of the reported counts

Every count in §2.2, §3.1, and §6.3 was re-derived from the repository rather than quoted (Table 7).

Table 7: Verification of the counts reported in this paper.

Claim	How verified	Result
Benchmark composition	counted from the 491 dataset files	142 miscompilation / 340 crash / 9 hang; 1,462 reproducers
Single-function-fix miscompilations	counted from dataset properties	103
IR reader fidelity	round-trip check over the whole dataset	1,462 ok, 0 mismatched, 0 crashed
Sample determinism	re-ran the selector, diffed against the committed manifest	identical 24-bug list, pool = 99
Sample diversity	tabulated the manifest	16 distinct components, 8 bugs per tier
Discordance floor	re-ran the power script	≥ 5 discordant bugs required
Power grid	re-ran and extended to $k = 1 \ldots 5$	matches the preregistered table
Window and no-op handling	read the repair runner	no-op guard, brace-balanced windows, rebalanced format requirement all present

Not verifiable in the analysis environment: anything requiring `alive-tv` or a built `opt`, which includes live reduction measurement and the integration test suite. No reduction measurements are reported in this document

for that reason.

B Branch provenance

Three branches were examined. `main` carries the vendored benchmark and the research framing document. `feat/counterexample-toolkit` adds the implementation — the three reducers, the oracle, the IR model, the structured renderer, the condition matrix, the runner, and the test suite. `feat/h100-sweep` builds on it with server configuration, operating and runbook documentation, a native setup path, sweep triage and patch-inspection tooling, a resume test, and the decision log this document draws on. A fourth branch, `feat/e2e-bootstrap`, was merged earlier and carried the first end-to-end build verification. All method content here is taken from `feat/h100-sweep`, the most complete state.

References

[1] N. P. Lopes, J. Lee, C.-K. Hur, Z. Liu, and J. Regehr. Alive2: Bounded translation validation for LLVM. *PLDI*, 2021.

[2] A. Zeller and R. Hildebrandt. Simplifying and isolating failure-inducing input. *IEEE TSE*, 28(2), 2002.

[3] Survey of automated program repair. `ref_papers/automated_program_repair.pdf`.

[4] Conversational automated program repair. `ref_papers/chat_repair.pdf`.

[5] Reduction-guided repair feedback. `ref_papers/reducefix.pdf`.

[6] LLM agents for automated program repair. `ref_papers/repair_agent.pdf`.

[7] Agentic harnesses for software repair. `ref_papers/agentic_harness.pdf`.